

Escuela Politécnica Nacional

Nombre: Kevin Eduardo Garcia Rodriguez

Tema: [Tarea 08] Minimos Cuadrados

Repositorio GIT: <https://github.com/Nattyrd/Metodos-Numericos-2025B>

1. Dados los datos

xi	4.0	4.2	4.5	4.7	5.1	5.5	5.9	6.3	6.8	7.1
yi	102.56	130.11	113.18	142.05	167.53	195.14	224.87	256.73	299.50	326.72

a. Construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.

```
In [3]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Datos
x = np.array([4.0, 4.2, 4.5, 4.7, 5.1, 5.5, 5.9, 6.3, 6.8, 7.1])
y = np.array([102.56, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 299.50, 326.72])

# Función para calcular el error cuadrático medio
def error_cuadratico_medio(y_real, y_predicho):
    return np.mean((y_real - y_predicho) ** 2)

# Ajuste polinómico de grado 1
coef1 = np.polyfit(x, y, 1)
y1 = np.polyval(coef1, x)
error1 = error_cuadratico_medio(y, y1)

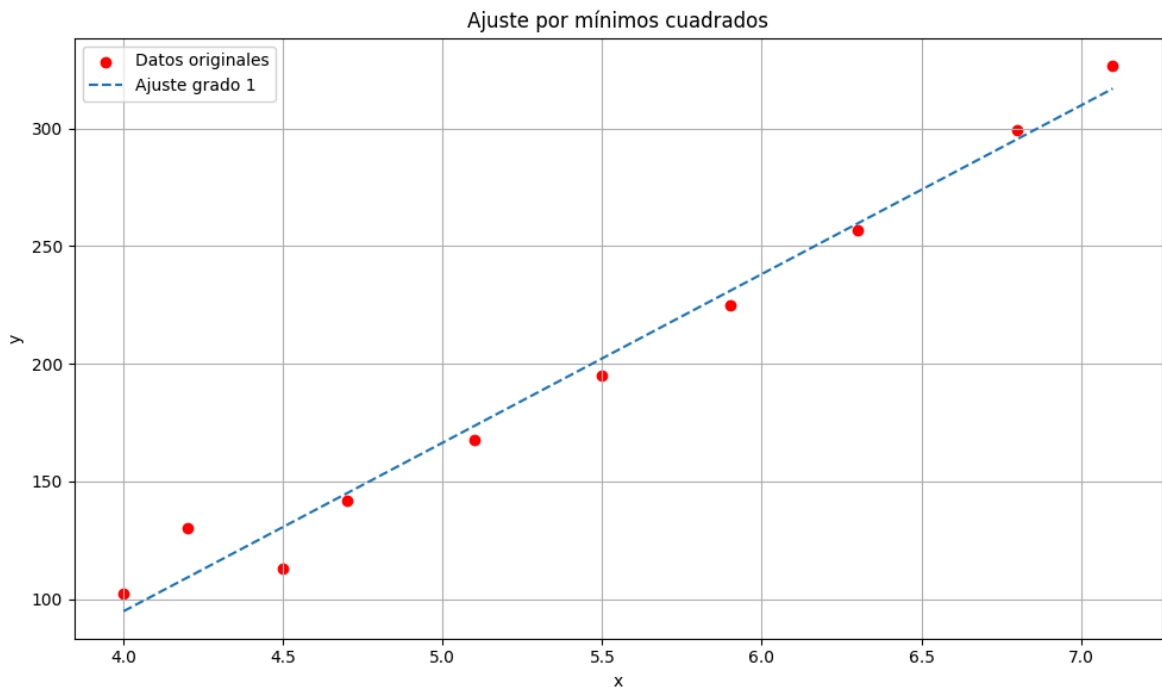
# Mostrar resultados
print("Ajuste polinomial de grado 1:", coef1)
print("Error cuadrático medio (grado 1):", error1)

# Gráfico
plt.figure(figsize=(10, 6))
plt.scatter(x, y, color='red', label='Datos originales')

# Línea de ajuste grado 1
x_line = np.linspace(min(x), max(x), 200)
plt.plot(x_line, np.polyval(coef1, x_line), linestyle='--', label='Ajuste grado 1')

# Personalización del gráfico
plt.title('Ajuste por mínimos cuadrados')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Ajuste polinomial de grado 1: [71.61024372 -191.57241853]
 Error cuadrático medio (grado 1): 105.88388862638901



b. Construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.

```
In [5]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Datos
x = np.array([4.0, 4.2, 4.5, 4.7, 5.1, 5.5, 5.9, 6.3, 6.8, 7.1])
y = np.array([102.56, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 290.11, 324.11])

# Función para calcular el error cuadrático medio
def error_cuadratico_medio(y_real, y_predicho):
    return np.mean((y_real - y_predicho) ** 2)

# Ajuste polinómico de grado 2
coef2 = np.polyfit(x, y, 2)
y2 = np.polyval(coef2, x)
error2 = error_cuadratico_medio(y, y2)

# Mostrar resultados
print("Ajuste polinomial de grado 2:", coef2)
print("Error cuadrático medio (grado 2):", error2)

# Gráfico
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Polinomio grado 2', color='black')

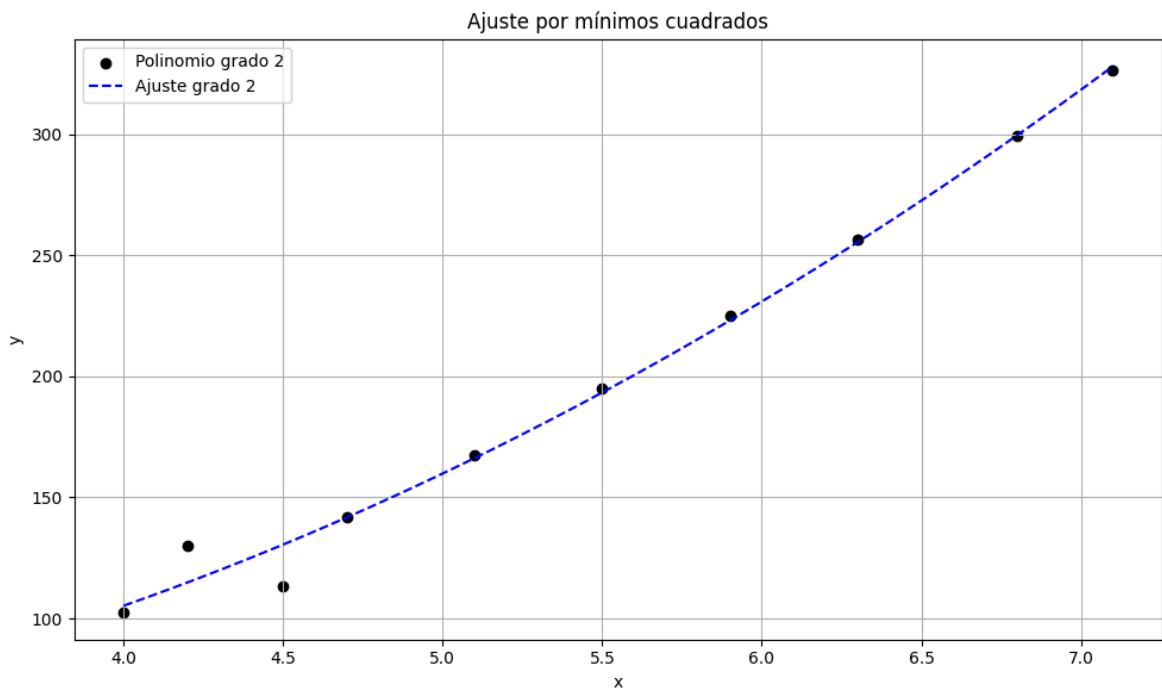
# Línea de ajuste grado 2
x_line = np.linspace(min(x), max(x), 200)
plt.plot(x_line, np.polyval(coef2, x_line), color='blue', linestyle='--', label='Ajuste grado 2')

# Personalización del gráfico
plt.title('Ajuste por mínimos cuadrados')
plt.xlabel('x')
plt.ylabel('y')
```

```
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Ajuste polinomial de grado 2: [8.21707232 -19.30860379 51.00078939]

Error cuadrático medio (grado 2): 55.16562001170238



c. Construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.

```
In [8]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Datos
x = np.array([4.0, 4.2, 4.5, 4.7, 5.1, 5.5, 5.9, 6.3, 6.8, 7.1])
y = np.array([102.56, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 290.00, 320.00])

# Función para calcular el error cuadrático medio
def error_cuadratico_medio(y_real, y_predicho):
    return np.mean((y_real - y_predicho) ** 2)

# Ajuste polinómico de grado 3
coef3 = np.polyfit(x, y, 3)
y3 = np.polyval(coef3, x)
error3 = error_cuadratico_medio(y, y3)

# Mostrar resultados
print("Ajuste polinomial de grado 3:", coef3)
print("Error cuadrático medio (grado 3):", error3)

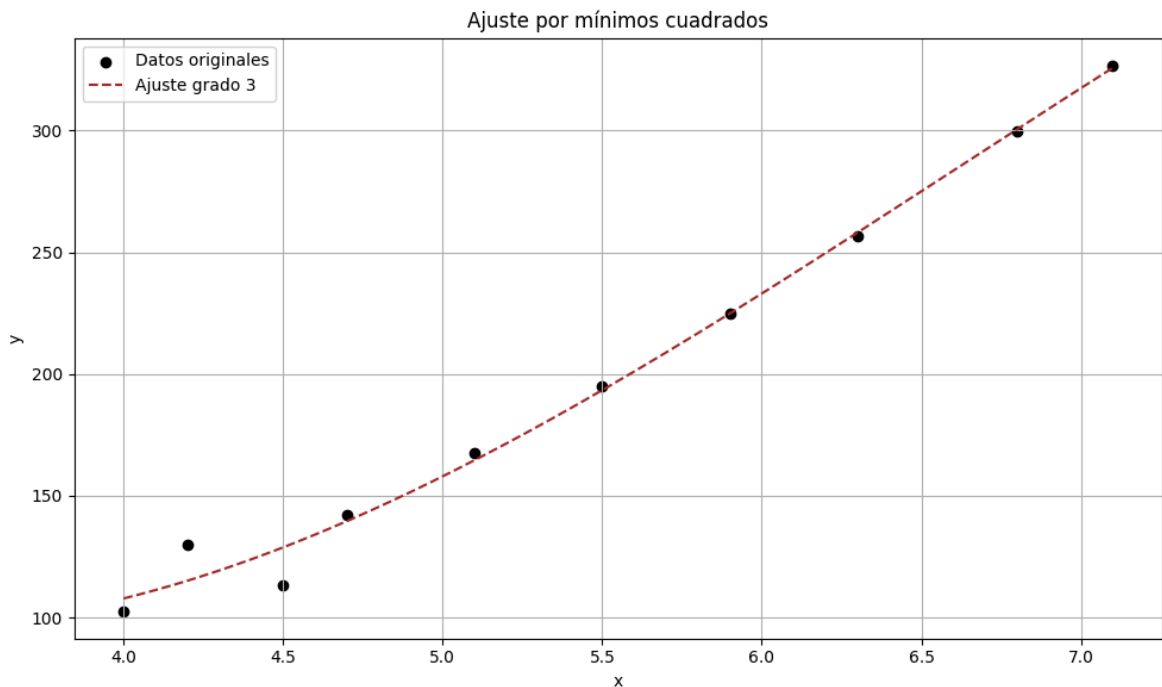
# Gráfico
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Datos originales', color='black')

# Curva ajustada (grado 3)
x_line = np.linspace(min(x), max(x), 200)
plt.plot(x_line, np.polyval(coef3, x_line), linestyle='--', color='brown', label='Ajuste grado 3')
```

```
# Personalización del gráfico
plt.title('Ajuste por mínimos cuadrados')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Ajuste polinomial de grado 3: [-2.60683872 51.56095694 -254.87478338 469.16326528]

Error cuadrático medio (grado 3): 51.83830647403031



d. Construya el polinomio por mínimos cuadrados de la forma be^{ax} y calcule el error.

```
In [9]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Datos
x = np.array([4.0, 4.2, 4.5, 4.7, 5.1, 5.5, 5.9, 6.3, 6.8, 7.1])
y = np.array([102.56, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 296.11, 324.87])

# Función para calcular el error cuadrático medio
def error_cuadratico_medio(y_real, y_predicho):
    return np.mean((y_real - y_predicho) ** 2)

# Ajuste exponencial: y = b * exp(a * x)
def modelo_exp(x, a, b):
    return b * np.exp(a * x)

params_exp, _ = curve_fit(modelo_exp, x, y)
y_exp = modelo_exp(x, *params_exp)
error_exp = error_cuadratico_medio(y, y_exp)

# Mostrar resultados
print("Ajuste exponencial: a =", params_exp[0], ", b =", params_exp[1])
print("Error exponencial:", error_exp)
```

```

# Graficar
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Datos originales', color='black')

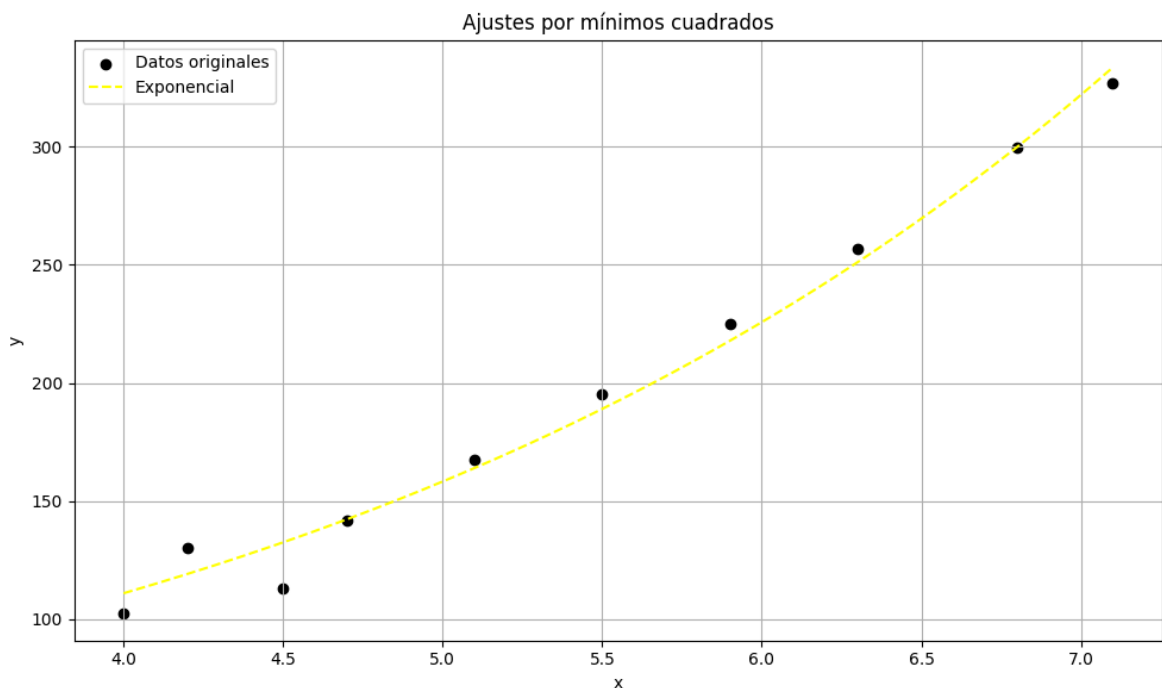
# Curvas ajustadas
x_line = np.linspace(min(x), max(x), 200)
plt.plot(x_line, modelo_exp(x_line, *params_exp), label='Exponencial', linestyle='dashed', color='yellow')

plt.title('Ajustes por mínimos cuadrados')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Ajuste exponencial: $a = 0.3549271013132163$, $b = 26.840771676946506$

Error exponencial: 74.36215913505171



e. Construya el polinomio por mínimos cuadrados de la forma bx^a y calcule el error.

```

In [6]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Datos
x = np.array([4.0, 4.2, 4.5, 4.7, 5.1, 5.5, 5.9, 6.3, 6.8, 7.1])
y = np.array([102.56, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 290.00, 320.00])

# Función para calcular el error cuadrático medio
def error_cuadratico_medio(y_real, y_predicho):
    return np.mean((y_real - y_predicho) ** 2)

# Ajuste potencia: y = b * x^a
def modelo_pot(x, a, b):
    return b * x**a

```

```

params_pot, _ = curve_fit(modelo_pot, x, y)
y_pot = modelo_pot(x, *params_pot)
error_pot = error_cuadratico_medio(y, y_pot)

# Mostrar resultados

print("Ajuste potencia: a =", params_pot[0], ", b =", params_pot[1])
print("Error potencia:", error_pot)

# Graficar
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Datos originales', color='black')

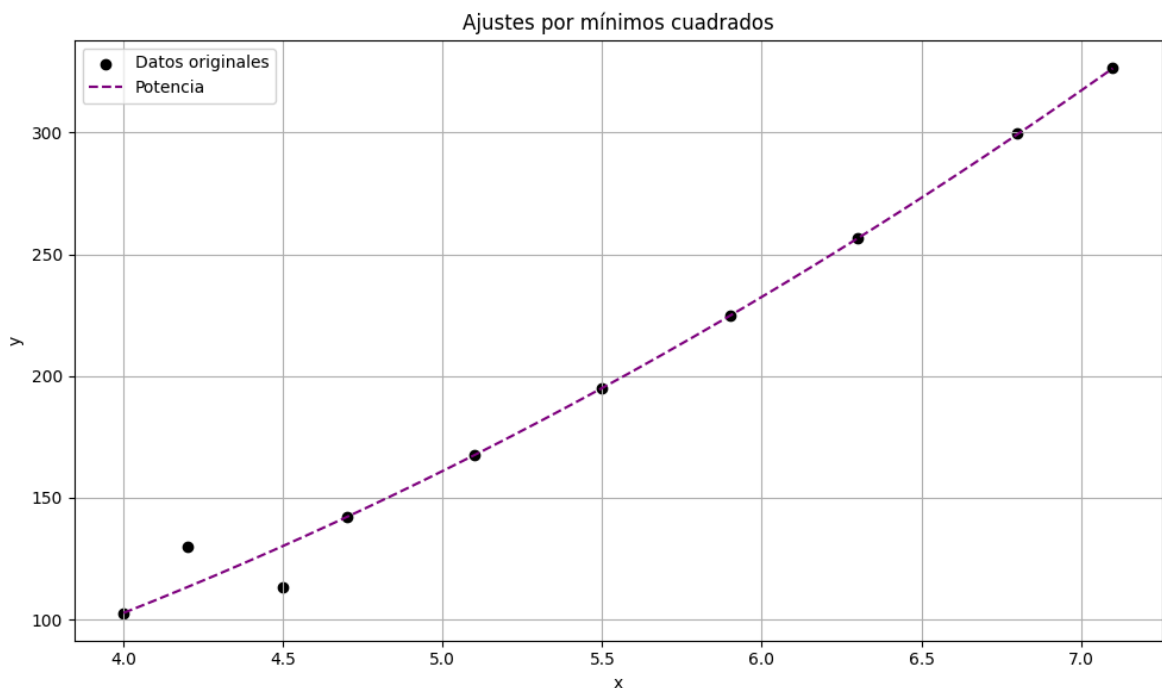
# Curvas ajustadas
x_line = np.linspace(min(x), max(x), 200)
plt.plot(x_line, modelo_pot(x_line, *params_pot), label='Potencia', linestyle='--')

plt.title('Ajustes por mínimos cuadrados')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Ajuste potencia: a = 2.0151903407063725 , b = 6.28396610625708

Error potencia: 57.25817542056741



2. Repita el ejercicio 5 para los siguientes datos.

xi	0.2	0.3	0.6	0.9	1.1	1.3	1.4	1.6
yi	0.050446	0.098426	0.33277	0.72660	1.0972	1.5697	1.8487	2.5015

a. Construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.

```

In [10]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

```

```

# Datos
x = np.array([0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6])
y = np.array([0.050446, 0.098426, 0.33277, 0.72660, 1.0972, 1.5697, 1.8487, 2.501])

# Función para calcular el error cuadrático medio
def error_cuadratico_medio(y_real, y_predicho):
    return np.mean((y_real - y_predicho) ** 2)

# Ajuste polinómico de grado 1, 2 y 3
coef1 = np.polyfit(x, y, 1)

y1 = np.polyval(coef1, x)
error1 = error_cuadratico_medio(y, y1)

# Mostrar resultados
print("Ajuste polinomial grado 1:", coef1)
print("Error grado 1:", error1)

# Graficar
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Datos originales', color='orange')

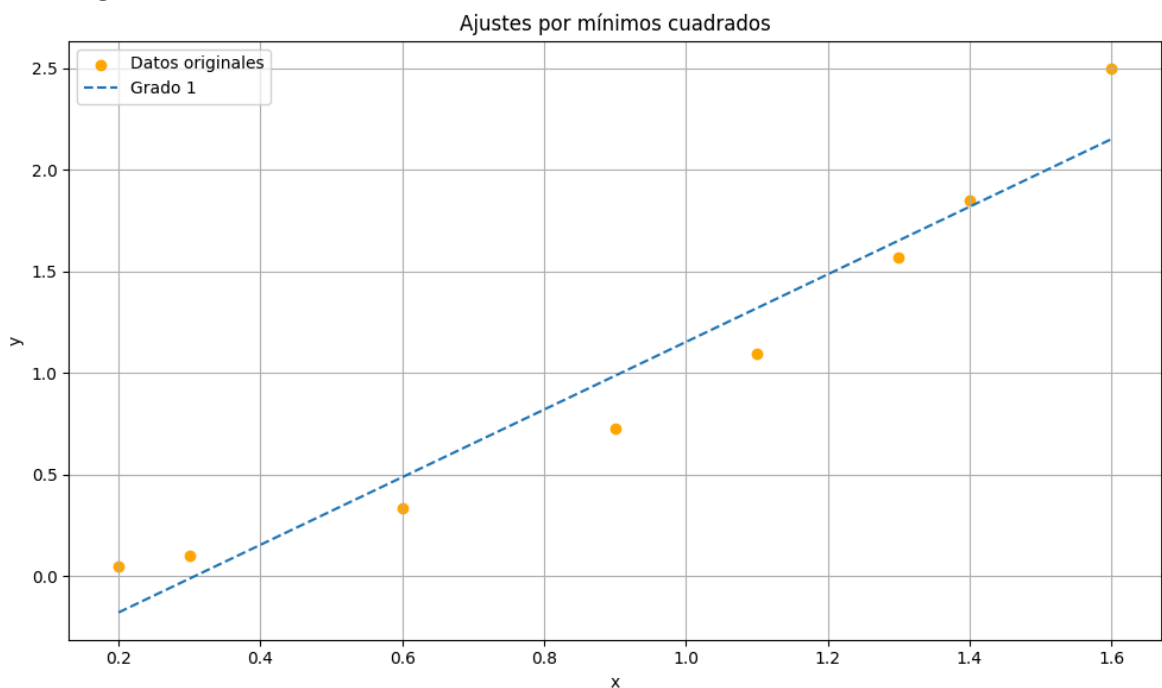
# Curvas ajustadas
x_line = np.linspace(min(x), max(x), 200)
plt.plot(x_line, np.polyval(coef1, x_line), label='Grado 1', linestyle='--')

plt.title('Ajustes por mínimos cuadrados')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Ajuste polinomial grado 1: [1.66554008 -0.51245682]

Error grado 1: 0.041948731969935996



b. Construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.

```

In [2]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Datos
x = np.array([0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6])
y = np.array([0.050446, 0.098426, 0.33277, 0.72660, 1.0972, 1.5697, 1.8487, 2.501])

# Función para calcular el error cuadrático medio
def error_cuadratico_medio(y_real, y_predicho):
    return np.mean((y_real - y_predicho) ** 2)

# Ajuste polinómico de grado 1, 2 y 3
coef2 = np.polyfit(x, y, 2)

y2 = np.polyval(coef1, x)
error2 = error_cuadratico_medio(y, y1)

# Mostrar resultados
print("Ajuste polinomial grado 1:", coef2)
print("Error grado 2:", error2)

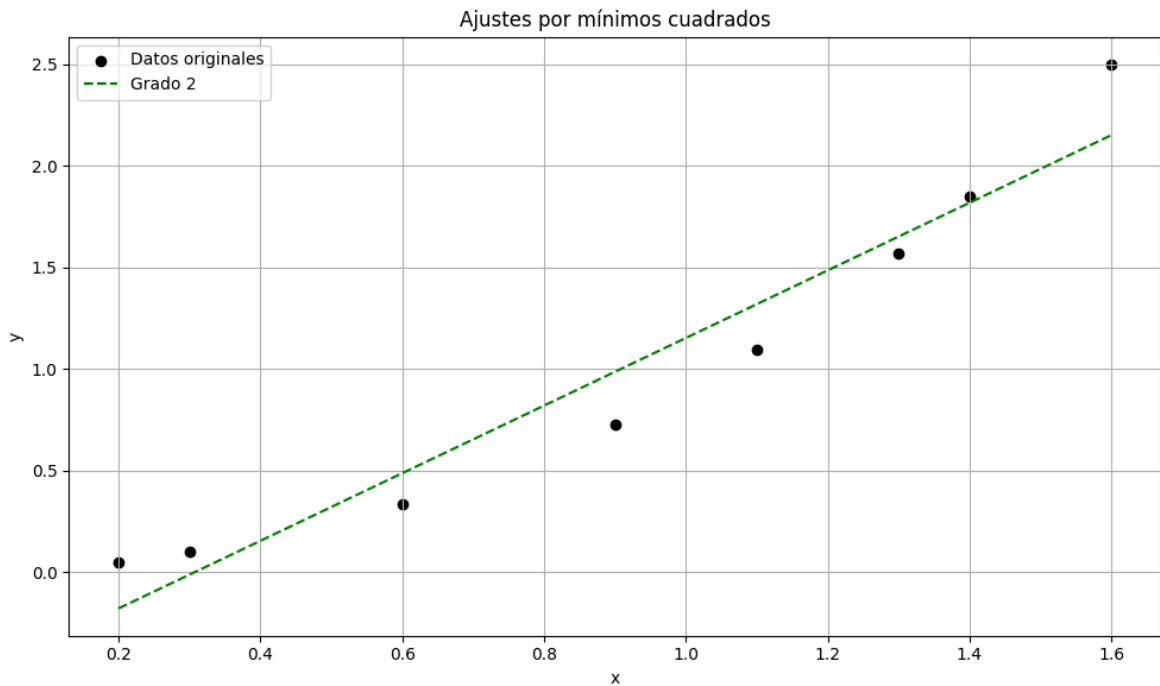
# Graficar
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Datos originales', color='black')

# Curvas ajustadas
x_line = np.linspace(min(x), max(x), 200)
plt.plot(x_line, np.polyval(coef1, x_line), label='Grado 2', linestyle='--', color='red')

plt.title('Ajustes por mínimos cuadrados')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Ajuste polinomial grado 1: [1.12942387 -0.31140346 0.08514393]
 Error grado 2: 0.041948731969935996



c. Construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.

```
In [11]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Datos
x = np.array([0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6])
y = np.array([0.050446, 0.098426, 0.33277, 0.72660, 1.0972, 1.5697, 1.8487, 2.501])

# Función para calcular el error cuadrático medio
def error_cuadratico_medio(y_real, y_predicho):
    return np.mean((y_real - y_predicho) ** 2)

# Ajuste polinómico de grado 1, 2 y 3
coef3 = np.polyfit(x, y, 3)

y3 = np.polyval(coef3, x)

error3 = error_cuadratico_medio(y, y3)

print("Ajuste polinomial grado 3:", coef3)
print("Error grado 3:", error3)

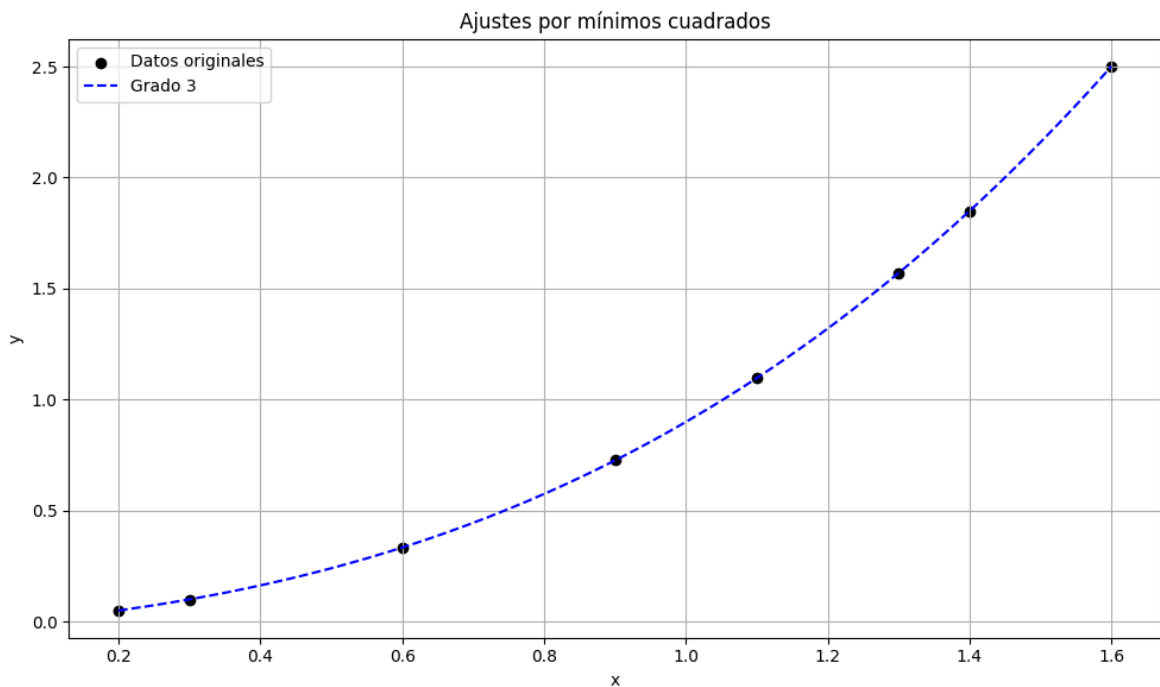
# Graficar
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Datos originales', color='black')

# Curvas ajustadas
x_line = np.linspace(min(x), max(x), 200)
plt.plot(x_line, np.polyval(coef3, x_line), label='Grado 3', linestyle='--', color='green')

plt.title('Ajustes por mínimos cuadrados')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
```

```
plt.grid(True)
plt.tight_layout()
plt.show()
```

Ajuste polinomial grado 3: [0.2662081 0.40293221 0.24838578 -0.0184014]
 Error grado 3: 6.343339444535476e-07



d. Construya el polinomio por mínimos cuadrados de la forma be^{ax} y calcule el error.

```
In [12]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Datos
x = np.array([0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6])
y = np.array([0.050446, 0.098426, 0.33277, 0.72660, 1.0972, 1.5697, 1.8487, 2.501])

# Función para calcular el error cuadrático medio
def error_cuadratico_medio(y_real, y_predicho):
    return np.mean((y_real - y_predicho) ** 2)

# Ajuste exponencial: y = b * exp(a * x)
def modelo_exp(x, a, b):
    return b * np.exp(a * x)

params_exp, _ = curve_fit(modelo_exp, x, y)
y_exp = modelo_exp(x, *params_exp)
error_exp = error_cuadratico_medio(y, y_exp)

# Mostrar resultados
print("Ajuste exponencial: a =", params_exp[0], ", b =", params_exp[1])
print("Error exponencial:", error_exp)

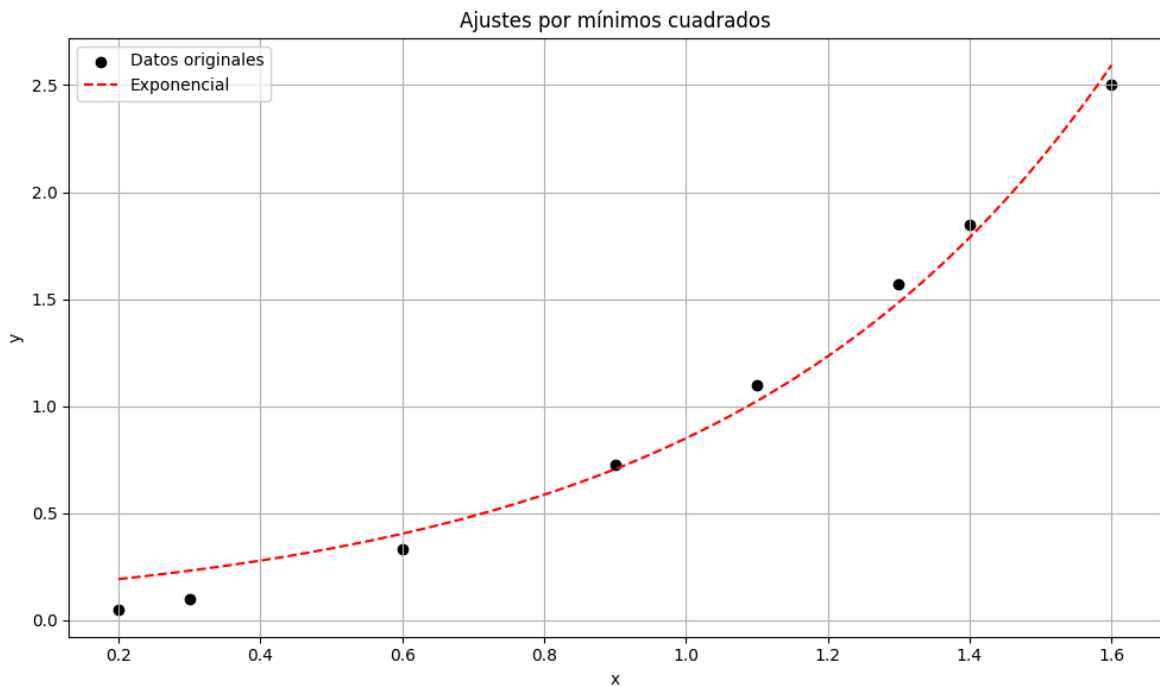
# Graficar
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Datos originales', color='black')
```

```
# Curvas ajustadas
x_line = np.linspace(min(x), max(x), 200)
plt.plot(x_line, modelo_exp(x_line, *params_exp), label='Exponencial', linestyle='--')

plt.title('Ajustes por mínimos cuadrados')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Ajuste exponencial: $a = 1.8583147785155039$, $b = 0.13258632607461648$

Error exponencial: 0.008498467250197317



e. Construya el polinomio por mínimos cuadrados de la forma bx^a y calcule el error.

```
In [5]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Datos
x = np.array([0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6])
y = np.array([0.050446, 0.098426, 0.33277, 0.72660, 1.0972, 1.5697, 1.8487, 2.501])

# Función para calcular el error cuadrático medio
def error_cuadratico_medio(y_real, y_predicho):
    return np.mean((y_real - y_predicho) ** 2)

# Ajuste potencia: y = b * x^a
def modelo_pot(x, a, b):
    return b * x**a

params_pot, _ = curve_fit(modelo_pot, x, y)
y_pot = modelo_pot(x, *params_pot)
error_pot = error_cuadratico_medio(y, y_pot)

# Mostrar resultados
print("Ajuste potencia: a =", params_pot[0], ", b =", params_pot[1])
```

```

print("Error potencia:", error_pot)

# Graficar
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Datos originales', color='black')

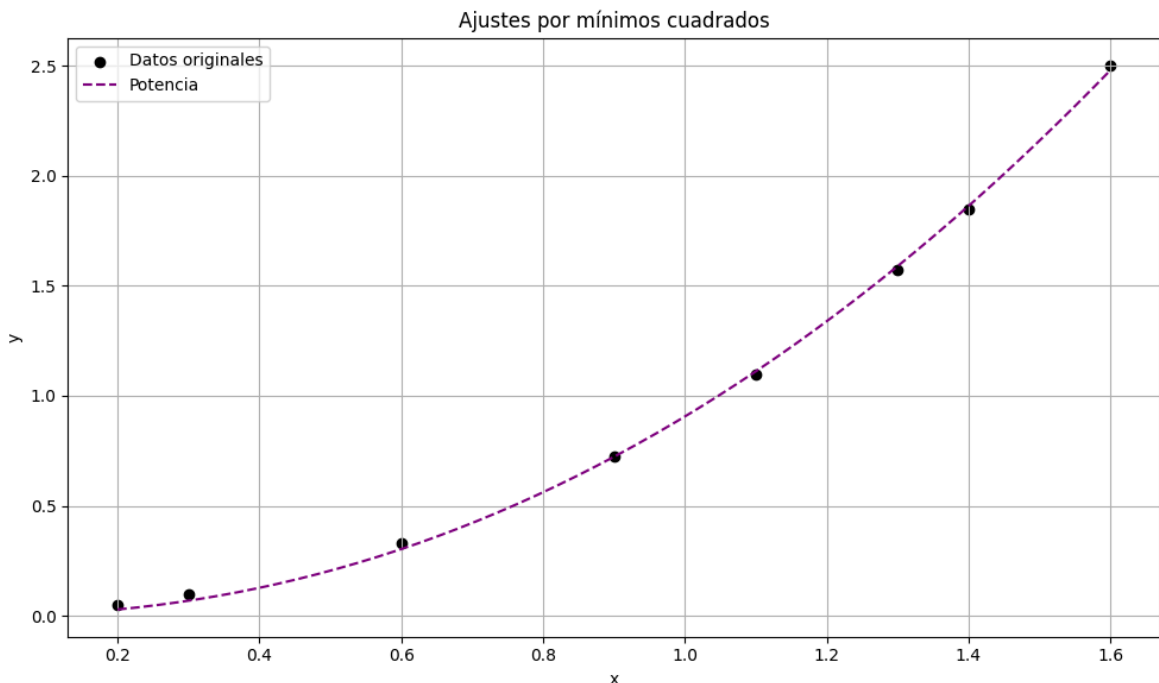
# Curvas ajustadas
x_line = np.linspace(min(x), max(x), 200)
plt.plot(x_line, modelo_pot(x_line, *params_pot), label='Potencia', linestyle='--')

plt.title('Ajustes por mínimos cuadrados')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Ajuste potencia: $a = 2.142772882974873$, $b = 0.9055224505274626$

Error potencia: 0.00043594151427458805



3. La siguiente tabla muestra los promedios de puntos del colegio de 20 especialistas en matemáticas y ciencias computacionales, junto con las calificaciones que recibieron estos estudiantes en la parte de matemáticas de la prueba ACT (Programa de Pruebas de Colegios Americanos) mientras estaban en secundaria. Grafique estos datos y encuentre la ecuación de la recta por mínimos cuadrados para estos datos.

Puntuación ACT	Promedio de puntos	Puntuación ACT	Promedio de puntos
28	3.84	29	3.75
25	3.21	28	3.65
28	3.23	27	3.87
27	3.63	29	3.75
28	3.75	21	1.66

Puntuación ACT	Promedio de puntos	Puntuación ACT	Promedio de puntos
33	3.20	28	3.12
28	3.41	28	2.96
29	3.38	26	2.92
23	3.53	30	3.10
27	2.03	24	2.81

```
In [6]: import numpy as np
import matplotlib.pyplot as plt

# Datos del ejercicio (ACT score y promedio de puntos)
x = np.array([
    28, 25, 28, 27, 27, 33, 28, 28, 29, 27,
    29, 28, 27, 29, 21, 28, 26, 30, 26, 24
])
y = np.array([
    3.84, 3.21, 3.23, 3.63, 3.75, 3.20, 3.41, 3.38, 3.53, 2.03,
    3.75, 3.65, 3.87, 3.75, 3.12, 2.96, 2.92, 3.10, 2.88, 2.81
])

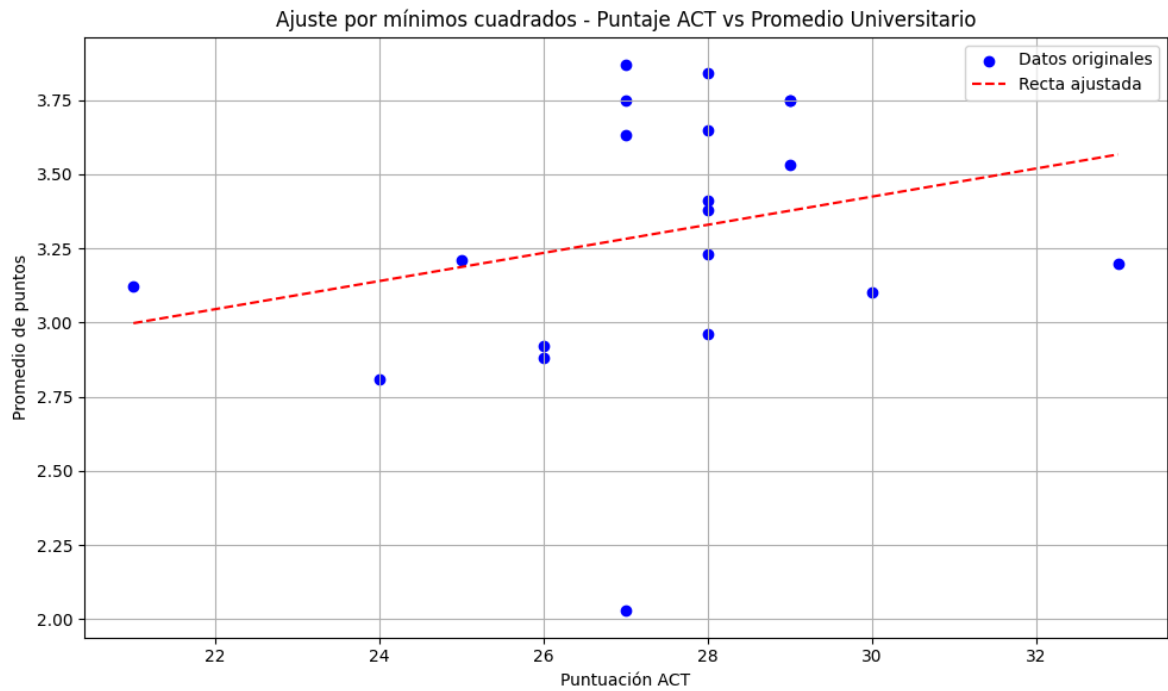
# Ajuste por mínimos cuadrados (polinomio de grado 1)
coef = np.polyfit(x, y, 1)
a, b = coef

# Evaluar la recta
x_line = np.linspace(min(x), max(x), 200)
y_line = np.polyval(coef, x_line)

# Mostrar ecuación de la recta
print(f"Ecuación ajustada: y = {a:.4f}x + {b:.4f}")

# Gráfica
plt.figure(figsize=(10,6))
plt.scatter(x, y, color='blue', label='Datos originales')
plt.plot(x_line, y_line, color='red', linestyle='--', label='Recta ajustada')
plt.title('Ajuste por mínimos cuadrados - Puntaje ACT vs Promedio Universitario')
plt.xlabel('Puntuación ACT')
plt.ylabel('Promedio de puntos')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Ecuación ajustada: $y = 0.0474x + 2.0022$



4. El siguiente conjunto de datos, presentado al Subcomité Antimonopolio del Senado, muestra las características comparativas de supervivencia durante un choque de automóviles de diferentes clases. Encuentre la recta por mínimos cuadrados que aproxima estos datos (la tabla muestra el porcentaje de vehículos que participaron en un accidente en los que la lesión más grave fue fatal o seria).

Tipo	Peso promedio	Porcentaje de presentación
1. Regular lujoso doméstico	4800 lb	3.1
2. Regular intermediario doméstico	3700 lb	4.0
3. Regular económico doméstico	3400 lb	5.2
4. Compacto doméstico	2800 lb	6.4
5. Compacto extranjero	1900 lb	9.6

```
In [14]: import numpy as np
import matplotlib.pyplot as plt

# Datos del ejercicio
peso = np.array([4800, 3700, 3400, 2800, 1900]) # en libras
porcentaje = np.array([3.1, 4.0, 5.2, 6.4, 9.6]) # porcentaje de presentación

# Ajuste por mínimos cuadrados (grado 1)
coef = np.polyfit(peso, porcentaje, 1)
a, b = coef

# Evaluación de la recta
peso_line = np.linspace(min(peso), max(peso), 200)
porcentaje_line = np.polyval(coef, peso_line)

# Mostrar la ecuación ajustada
print(f"Ecuación ajustada: y = {a:.5f}x + {b:.2f}")

# Gráfico
```

```
plt.figure(figsize=(10, 6))
plt.scatter(peso, porcentaje, color='brown', label='Datos observados')
plt.plot(peso_line, porcentaje_line, color='black', linestyle='--', label='Recta')
plt.title('Peso promedio vs Porcentaje de accidentes graves')
plt.xlabel('Peso (libras)')
plt.ylabel('Porcentaje de presentación (%)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Ecuación ajustada: $y = -0.00225x + 13.15$

